

Visão Geral

“Uma boa linguagem de programação é um universo conceitual para pensar em programação.”

Alan Perlis

VISÃO GERAL DO CAPÍTULO

1.1	PRINCÍPIOS	2
1.2	PARADIGMAS	3
1.3	TÓPICOS ESPECIAIS	5
1.4	UMA BREVE HISTÓRIA	6
1.5	SOBRE O PROJETO DE LINGUAGEM	11
1.6	COMPILADORES E MÁQUINAS VIRTUAIS	18
1.7	RESUMO	20
	EXERCÍCIOS	21

Da mesma forma que as nossas linguagens naturais, as linguagens de programação facilitam a expressão e a comunicação de idéias entre pessoas. Entretanto, linguagens de programação diferem das linguagens naturais de duas maneiras importantes. Em primeiro lugar, linguagens de programação também permitem a comunicação de idéias entre pessoas e computadores. Em segundo lugar, as linguagens de programação possuem um domínio de expressão mais reduzido do que o das linguagens naturais. Isso quer dizer que elas facilitam apenas a comunicação de idéias *computacionais*. Assim, uma linguagem de

programação deve satisfazer requisitos diferentes daqueles de uma linguagem natural. Este texto explora esses requisitos e as alternativas de projeto de linguagens que eles evocam.

Neste estudo, identificamos as muitas semelhanças entre linguagens de programação e linguagens naturais. Também examinamos as diferenças fundamentais que são impostas pela configuração computacional na qual um programa deve funcionar. Examinamos as características das linguagens de programação tanto abstratamente quanto ativamente. Isso significa que combinamos um tratamento conceitualmente rico de projeto de linguagens de programação com um estudo prático em laboratório sobre como esses conceitos afetam projetistas de linguagens e programadores em uma ampla faixa de domínios de aplicações.

Este estudo é importante porque os alunos de ciência da computação de hoje serão os projetistas e usuários das linguagens de programação de amanhã. Para se tornar um projetista e usuário de linguagens bem informado você precisará compreender as linguagens de forma ampla – suas características, seus pontos fortes e fracos em uma ampla gama de estilos de programação e suas aplicações. Conhecer uma linguagem e o domínio de sua aplicação não fornece tal amplitude de conhecimento. Este livro lhe ajudará a obter tal amplitude.

1.1 PRINCÍPIOS

Projetistas de linguagens possuem um vocabulário básico sobre a estrutura, o significado e as preocupações pragmáticas dessas ferramentas que os auxiliam a entender como as linguagens funcionam. Esse vocabulário se divide em três categorias principais que chamamos de *princípios* de projeto de linguagens:

- Sintaxe
- Nomes e tipos
- Semântica

Muitos dos conceitos dessas categorias são emprestados da lingüística e da matemática, como veremos a seguir. Juntas, essas categorias fornecem um foco organizacional para os Capítulos básicos 2, 4, 5, 7 e 9, respectivamente. Mais profundidade de estudo em cada categoria é apresentada nos capítulos associados (3, 6, 8, 10 e 11), conforme explicado a seguir.

Sintaxe A *sintaxe* de uma linguagem descreve o que constitui um programa estruturalmente correto. Ela responde a muitas questões. Qual é a gramática para se escrever programas na linguagem? Qual é o conjunto básico de palavras e símbolos que os programadores usam para escrever programas estruturalmente corretos?

Veremos que a maioria da estrutura sintática de linguagens modernas de programação é definida com o uso de um formalismo lingüístico denominado *gramática livre de contexto*. Outros elementos de sintaxe estão fora do domínio de gramáticas livres de contexto e são definidos por outros meios. Um tratamento cuidadoso da sintaxe das linguagens de programação aparece no Capítulo 2.

Um estudo de sintaxe de linguagens traz muitas questões. Como um compilador analisa a sintaxe de um programa? Como os erros de sintaxe são detectados? Como uma gramática livre de contexto facilita o desenvolvimento de um analisador sintático? Essas questões mais profundas sobre sintaxe são abordadas no Capítulo 3.

Nomes e Tipos O vocabulário de uma linguagem de programação inclui um conjunto de regras cuidadosamente projetado para nomear entidades – variáveis, funções, classes, parâmetros e assim por diante. Nomes de entidades também têm outras propriedades durante a vida de um programa, como seu escopo, visibilidade e ligação. O estudo de nomes em linguagens de programação e seu impacto sobre a sintaxe e a semântica de um programa é o assunto do Capítulo 4.

Os *tipos* de uma linguagem denotam os tipos de valores que os programas podem manipular: tipos simples, tipos estruturados e tipos mais complexos. Entre os tipos simples estão valores inteiros, números decimais, caracteres e booleanos. Tipos estruturados incluem strings de caracteres, listas, árvores e tabelas hash. Tipos mais complexos incluem funções e classes. Os tipos são discutidos em maior extensão no Capítulo 5.

Um sistema de tipos permite ao programador entender e implementar apropriadamente operações sobre valores de diversos tipos. Um sistema de tipos especificado cuidadosamente permite ao compilador executar uma rigorosa verificação de tipos em um programa antes da sua execução, evitando assim erros em tempo de execução que podem ocorrer devido a operandos de tipos inapropriados. A especificação completa e a implementação de um sistema de tipos é o foco de um estudo mais aprofundado no Capítulo 6.

Semântica O significado de um programa é definido pela sua *semântica*, ou seja, quando um programa é executado, o efeito de cada comando sobre os valores das variáveis no programa é dado pela semântica da linguagem. Assim, quando escrevemos um programa, devemos entender idéias básicas como o efeito exato que uma atribuição possui sobre as variáveis do programa. Se tivermos um modelo semântico que seja independente de alguma plataforma específica, podemos aplicá-lo a uma diversidade de máquinas nas quais essa linguagem possa ser implementada. Estudamos semântica no Capítulo 7.

A implementação de semântica em tempo de execução também é interessante em um estudo mais profundo de semântica. Como um interpretador funciona e qual a conexão entre ele e a especificação da semântica de uma linguagem? Essas questões mais profundas são estudadas no Capítulo 8.

As funções representam o elemento chave da abstração procedural em qualquer linguagem. Uma compreensão da semântica da definição e chamada de funções é fundamental em qualquer estudo de linguagens de programação. A implementação de funções também requer uma compreensão dos elementos estáticos e dinâmicos de memória, incluindo a *pilha de tempo de execução*. A pilha também nos auxilia a entender outras idéias como o escopo de um nome e o tempo de vida de um objeto. Esses tópicos são tratados no Capítulo 9.

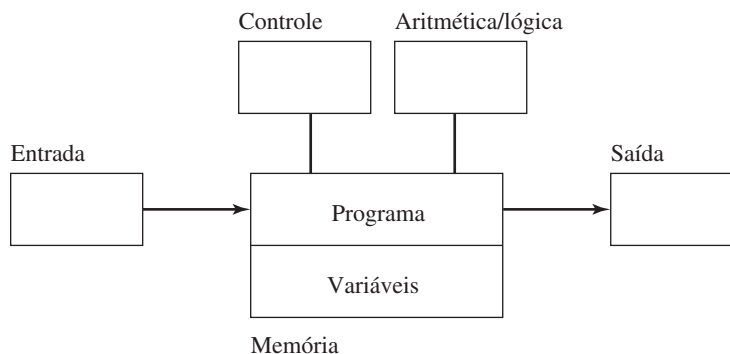
A implementação da pilha de chamadas e retornos de funções é um tópico central que merece estudo mais aprofundado. Além disso, estratégias para o gerenciamento de outra área de memória chamada de *heap* são importantes para a compreensão de objetos dinâmicos como matrizes. Técnicas de gerenciamento de *heap*, chamadas “coleta de lixo”, estão muito relacionadas à implementação destes objetos dinâmicos. A pilha e o *heap* são estudados em detalhes nos Capítulos 10 e 11, respectivamente.

1.2 PARADIGMAS

De modo geral, pensamos em um “paradigma” como um padrão de pensamento que guia um conjunto de atividades relacionadas. Um paradigma de programação é um padrão de resolução de problemas que se relaciona a um determinado gênero de programas e linguagens. Quatro paradigmas de programação distintos e fundamentais evoluíram nas últimas três décadas:

- Programação imperativa
- Programação orientada a objeto
- Programação funcional
- Programação lógica

Algumas linguagens de programação são intencionalmente projetadas para suportar mais de um paradigma. Por exemplo, C++ é uma linguagem imperativa e orientada



| Figura 1.1 O modelo computacional de von Neumann-Eckert

a objeto, enquanto a linguagem experimental Leda (Budd, 1995) é projetada para suportar os paradigmas de programação imperativa, orientada a objeto, funcional e lógica. Essas linguagens são reminiscências de trabalhos anteriores (notadamente PL/I, Algol 68 e Ada) para projetar uma única linguagem que fosse mais de propósito geral do que outras linguagens de programação do seu tempo. Com exceção de C++, esses esforços falharam ao tentar atrair interesse sustentado.

Programação Imperativa A *programação imperativa* é o paradigma mais antigo, já que está fundamentado no modelo computacional clássico de “von Neumann-Eckert” (veja a Figura 1.1). Nesse modelo, tanto o programa quanto as suas variáveis são armazenados juntos, e o programa contém uma série de comandos para executar cálculos, atribuir valores a variáveis, obter entradas, produzir saídas ou redirecionar o controle para outro ponto nessa série de comandos.

A abstração procedural é um componente para a programação imperativa assim como as atribuições, os laços, as seqüências, os comandos condicionais e a manipulação de exceções. As linguagens de programação imperativa predominantes incluem Cobol, Fortran, C, Ada e Perl. O paradigma de programação imperativa é o assunto do Capítulo 12.

Programação Orientada a Objeto A *programação orientada a objeto (POO)* fornece um modelo no qual um programa é uma coleção de objetos que interagem entre si, passando mensagens que transformam seu estado. Neste sentido, a passagem de mensagens permite que objetos de dados se tornem ativos em vez de passivos. Essa característica ajuda a distinguir melhor a programação OO da imperativa. A classificação de objetos, herança e a passagem de mensagens são componentes fundamentais da programação OO. Linguagens orientadas a objetos importantes são Smalltalk, C++, Java e C#. A programação OO será estudada no Capítulo 13.

Programação Funcional A *programação funcional* modela um problema computacional como uma coleção de funções matemáticas, cada uma com um espaço de entrada (domínio) e resultado (faixa). Isso separa a programação funcional das linguagens que possuem o comando de atribuição. Por exemplo, o comando de atribuição não

$$x = x + 1$$

faz sentido na programação funcional ou na matemática.

As funções interagem e combinam entre si usando composição funcional, condições e recursão. Importantes linguagens de programação funcional são Lisp, Scheme, Haskell e ML. A programação funcional é discutida e ilustrada no Capítulo 14.

Programação Lógica A *programação lógica* (declarativa) permite a um programa modelar um problema declarando qual resultado o programa deve obter, em vez de como ele deve ser obtido. Às vezes, essas linguagens são chamadas de *baseadas em regras*, já que as declarações do programa se parecem mais com um conjunto de regras ou restrições sobre o problema, em vez de uma seqüência de comandos a serem executados.

Interpretar as declarações de um programa lógico cria um conjunto de todas as soluções possíveis para o problema que ele especifica. A programação lógica também fornece um veículo natural para se expressar o não-determinismo, o que é apropriado para problemas cujas especificações sejam incompletas. A principal linguagem de programação lógica é Prolog, e o paradigma de programação lógica é descrito no Capítulo 15.

1.3 TÓPICOS ESPECIAIS

Além desses quatro paradigmas, diversos tópicos essenciais para o projeto de linguagem de programação merecem abordagem extensiva neste livro-texto. Esses tópicos tendem a ser universais, na medida em que aparecem em dois ou mais dos paradigmas anteriores, em vez de em apenas um. Cada um deles é brevemente introduzido agora:

- Manipulação de eventos
- Concorrência
- Correção

Manipulação de Eventos A *manipulação de eventos* ocorre com programas que respondem a eventos gerados em uma ordem imprevisível. De certa forma, um programa orientado a evento é apenas um programa cujo comportamento é inteiramente determinado por questões de manipulação de eventos. A manipulação de eventos está frequentemente acoplada ao paradigma orientado a objeto (por exemplo, Java applets), embora ela ocorra dentro dos paradigmas imperativos também (por exemplo, Tcl/Tk). Os eventos se originam de ações dos usuários na tela (cliques de mouse ou pressionamentos de teclas, por exemplo) ou então de outras fontes (como leituras de sensores em um robô). Linguagens importantes que suportam a manipulação de eventos incluem Visual Basic, Java e Tcl/Tk. Esse tópico é apresentado no Capítulo 16.

Concorrência A *programação concorrente* pode ocorrer dentro do paradigma imperativo, orientado a objeto, funcional ou lógico. A concorrência ocorre quando o programa possui uma coleção de elementos assíncronos que podem compartilhar informações ou sincronizar-se entre si em intervalos de tempo. A concorrência também ocorre dentro de um processo individual como a execução paralela de diferentes iterações de um laço. Linguagens de programação concorrente incluem SR (Andrews e Olsson, 1993), Linda (Carriero e Gelenter, 1989) e High Performance Fortran (Adams e outros, 1997). A programação concorrente será estudada no Capítulo 17.

Correção A *correção de programa* é um assunto que, até recentemente, tem atraído interesse apenas acadêmico. Todavia, linguagens mais novas e recursos de linguagens estão evoluindo para suportar o projeto de programas demonstravelmente corretos em uma diversidade de domínios de aplicações. Um programa é *correto* se satisfaz sua especificação formal para todas as suas entradas possíveis. A prova de correção é um assunto

complexo, mas ferramentas de linguagem para tratamento formal da correção por programadores estão agora se tornando disponíveis. O sistema Spark/Ada (Barnes, 2003) e a Java Modeling Language (Leavens et al., 1998) fornecem bons exemplos. Introduzimos o tópico de correção de programas no Capítulo 18.

1.4 UMA BREVE HISTÓRIA

As primeiras linguagens de programação foram as linguagens de máquina e as linguagens *assembly* dos primeiros computadores, começando na década de 1940. Centenas de linguagens de programação e dialetos foram desenvolvidos desde então. A maioria teve tempo de vida e utilidade limitados, enquanto algumas tiveram amplo sucesso em um ou mais domínios de aplicações. Muitas desempenharam um papel importante na influência sobre o projeto de futuras linguagens.

Um resumo do desenvolvimento histórico de diversas linguagens de programação influentes aparece na Figura 1.2. Embora certamente não seja completa, a Figura 1.2 identifica alguns dos eventos e das tendências mais influentes. Cada seta na Figura 1.2 indica uma significativa influência do projeto de uma linguagem mais antiga para uma sucessora.

A década de 1950 marcou o início da era das “linguagens de ordem mais alta” (abreviadas em inglês como HOLs). Uma HOL se distingue de uma linguagem de máquina ou *assembly* porque seu estilo de programação é independente de alguma arquitetura de máquina específica. As primeiras linguagens de ordem mais alta foram Fortran, Cobol, Algol e Lisp. Fortran e Cobol sobreviveram e se desenvolveram bastante desde o seu surgimento no final da década de 1950. Essas linguagens tiveram muitos seguidores e possuem uma grande quantidade de código legado no qual os programadores de hoje realizam manutenção. Por outro lado, Lisp teve substancialmente seu uso diminuído e Algol desapareceu completamente.

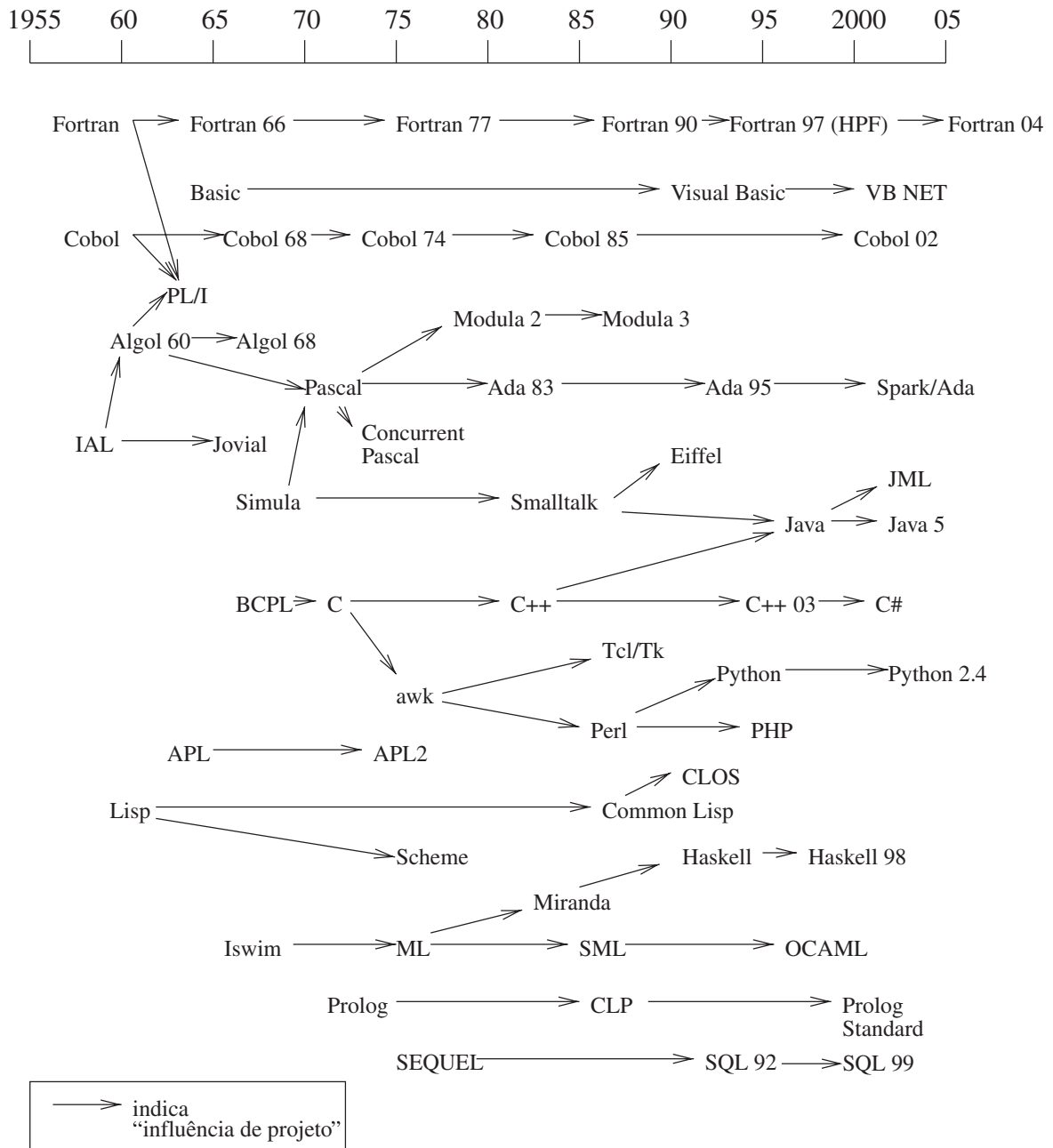
Contudo, os projetos inovadores dessas primeiras linguagens tiveram grande influência sobre suas sucessoras. Por exemplo, a demonstração de Fortran de que notação algébrica poderia ser traduzida em código eficiente agora é admitida como padrão, assim como a introdução da estrutura de registros em Cobol, o projeto de Pascal para a compilação em um passo e a demonstração de Algol de que uma gramática lingüística poderia definir formalmente sua sintaxe.

Talvez a maior motivação para o desenvolvimento de linguagens de programação nas últimas décadas tenha sido a rápida evolução da demanda de poder computacional e as novas aplicações por parte de uma grande e diversa comunidade de usuários. As seguintes comunidades de usuários podem reivindicar uma participação importante no cenário das linguagens de programação:

- Inteligência artificial
- Educação
- Ciência e engenharia
- Sistemas de informação
- Sistemas e redes
- World Wide Web

Os domínios dos problemas computacionais dessas comunidades são todos diferentes, assim como as principais linguagens de programação que se desenvolveram em torno delas. A seguir, esboçamos os principais objetivos computacionais e os projetos de linguagem que servem a cada uma dessas comunidades.

Inteligência Artificial A comunidade de programação de inteligência artificial está ativa desde o início da década de 1960. Esta comunidade se preocupa com o



| **Figura 1.2** Um Resumo da História das Linguagens de Programação

desenvolvimento de programas que modelam o comportamento da inteligência humana, a dedução lógica e a cognição. A manipulação de símbolos, as expressões funcionais e o projeto de sistemas de prova lógica têm sido os objetivos centrais desse trabalho em andamento.

Os paradigmas da *programação funcional* e da *programação lógica* se desenvolveram bastante por intermédio dos esforços dos programadores de inteligência artificial.

Linguagens de programação funcional proeminentes nos últimos anos incluem Lisp, Scheme, ML e Haskell. As linguagens de programação lógica proeminentes incluem Prolog e CLP.

A primeira linguagem de IA, Lisp (um acrônimo de “*List Processor*”), foi projetada por John McCarthy em 1960. A Figura 1.2 sugere que Lisp foi dominante nos primeiros anos e tornou-se menos dominante nos anos mais recentes. Entretanto, as características básicas de Lisp motivaram o desenvolvimento de linguagens mais recentes como Scheme, ML e Haskell. A forte relação entre Lisp e *lambda calculus* (um formalismo para modelar a natureza de funções matemáticas) fornece uma base matemática sólida para a posterior evolução desses sucessores. O *lambda calculus* e seu relacionamento com linguagens funcionais são mais explicados no Capítulo 14.

Na área de programação lógica, apenas uma linguagem, a Prolog, tem sido o ator principal, e a Prolog tem tido pouca influência sobre o projeto de linguagens em outras áreas de aplicação.

Educação Nas décadas de 1960 e 1970, diversas linguagens importantes foram projetadas com o objetivo principal de ensinar programação aos alunos. Por exemplo, Basic foi projetada na década de 1960 por John Kemeny para facilitar o aprendizado de programação por meio de *compartilhamento de tempo*, uma arquitetura na qual um único computador é conectado diretamente a diversos terminais de uma só vez. Cada usuário de terminal compartilha tempo no computador recebendo regularmente uma pequena “fatia de tempo” de poder computacional. Basic teve grande popularidade no decorrer dos anos, especialmente como uma linguagem de ensino em escolas de segundo grau e em programas de ciência de nível superior.

A linguagem Pascal, uma derivativa de Algol, foi projetada na década de 1970 com o propósito de ensinar programação. Pascal serviu durante muitos anos como a principal linguagem de ensino no currículo de ciência da computação em nível superior.

Durante a última década, essas linguagens têm sido substituídas em programas educacionais por linguagens de “apelo comercial” como C, C++ e Java. Essa mudança tem benefícios e desvantagens. Por um lado, aprender uma linguagem de apelo comercial fornece aos graduados uma ferramenta de programação que eles podem usar imediatamente quando se iniciarem na profissão de computação. Por outro lado, tal linguagem é inerentemente mais complexa e menos prática de aprender como primeira linguagem em um curso de graduação.

O recente surgimento de Python pode fornecer um meio pelo qual cursos introdutórios de ciência da computação podem retornar à simplicidade e se concentrar novamente no ensino dos primeiros princípios. Por exemplo, Python possui uma sintaxe e uma semântica mais transparentes, o que a torna mais acessível de ser bem entendida por um novato do que qualquer uma das suas alternativas com apelo comercial. Além disso, cursos introdutórios que utilizam Python parecem inserir uma variedade mais rica de tópicos de ciência da computação do que cursos usando C, C++ ou Java.

Ciência e Engenharia A comunidade de programação científica e de engenharia desempenhou um papel importante no início da história da computação e continua a desempenhar um papel importante atualmente. Os primeiros programas foram escritos na década de 1940 para prever as trajetórias balísticas durante a Segunda Guerra Mundial, usando a bem conhecida fórmula física que caracteriza corpos em movimento. Esses programas foram escritos primeiro em linguagem de máquina e *assembly*, por matemáticos especialmente treinados.

Uma força motriz importante por trás de aplicações científicas e de engenharia por toda a história é a necessidade de se obter o maior poder de processamento possível. O poder de processamento dos supercomputadores de hoje é medido em *teraflops* (trilhões

de operações de ponto flutuante por segundo), e o líder atual trabalha a uma velocidade de 280 teraflops executando um teste de desempenho-padrão chamado LINPAK (veja mais informações em www.top500.org). Muitas das aplicações científicas e de engenharia de hoje são modelos de sistemas naturais complexos em áreas como a bioinformática e as ciências da terra e atmosféricas.

A primeira linguagem de programação científica, Fortran I, foi projetada por John Backus na IBM em 1954 (Backus et al., 1954). O acrônimo “Fortran” é uma abreviação de “*Formula Translator*” (Tradutor de Fórmulas). Fortran é provavelmente a linguagem de programação científica mais usada atualmente.

Versões iniciais de Fortran tiveram muitos problemas, todavia. O mais difícil era a questão da consistência – o mesmo programa em Fortran era executado de forma diferente em máquinas diferentes ou então nem era executado. Esses problemas geraram diversos esforços novos. Um destes produziu a linguagem Algol, abreviação de “*Algorithmic Language*” (Linguagem Algorítmica), que foi projetada por um comitê internacional em 1959. O principal objetivo do projeto de Algol era fornecer uma linguagem mais bem definida que Fortran tanto para a computação quanto para a apresentação de algoritmos científicos e matemáticos.

Algol foi chamada originalmente de “*International Algebraic Language*” (IAL – Linguagem Algébrica Internacional). A linguagem Jovial foi projetada por Jules Schwartz na década de 1960 para refinar e aumentar os recursos de IAL. Esse acrônimo significa “*Jules’ Own Version of the International Algebraic Language*” (Versão Própria de Jules da Linguagem Algébrica Internacional). *Jovial* foi amplamente usada em aplicações do Departamento de Defesa dos Estados Unidos.

Outra linguagem interessante denominada APL (abreviação de “*A Programming Language*” – Uma Linguagem de Programação) (Iverson, 1962) foi projetada por Kenneth Iverson na década de 1960 para facilitar a programação rápida de álgebra de matrizes e outros cálculos matemáticos. APL tinha um conjunto de caracteres estendidos que incluía operadores de matrizes de símbolo único que poderiam substituir o tédio da escrita de laços *for* na maioria dos casos. A proliferação de tais símbolos especiais requereu o projeto de um teclado especializado para facilitar a digitação de programas em APL. Tais programas ficaram conhecidos pela sua brevidade; o cálculo de uma matriz que requeria um laço *for* explícito em uma linguagem convencional só precisava de um único símbolo em APL. A brevidade da APL também foi sua maldição aos olhos de muitas pessoas, ou seja, a maioria dos programas em APL era tão sucinta que desafiava a compreensão de todos que não fossem técnicos capacitados. O custo do suporte a um conjunto de caracteres especializado e um teclado estilizado também contribuiu para o fim da APL.

Até hoje, a computação científica permanece como uma atividade central na história da programação e das linguagens de programação. O domínio de seus problemas se refere principalmente à execução de cálculos complexos de forma muito rápida e precisa. Os cálculos são definidos por modelos matemáticos que representam fenômenos científicos. Eles são implementados principalmente por meio do uso do paradigma de *programação imperativa*. Linguagens modernas de programação que são amplamente usadas na área da programação científica incluem Fortran 90 (Chamberland, 1995), C (Kernighan e Ritchie, 1988), C++ (Stroustrup, 1997) e High Performance Fortran (Adams e outros, 1997).

Quanto mais complexo o fenômeno científico se torna, maior a necessidade de linguagens de programação e computadores altamente sofisticados e paralelos. Assim, a *programação concorrente* é fortemente motivada pelas necessidades de tais aplicações científicas como a modelagem de sistemas climáticos e de fluxo do oceano. Algumas linguagens, como High Performance Fortran, suportam programação concorrente por meio da inclusão de recursos a uma linguagem básica amplamente usada (Fortran). Outras, como SR e Occam, são projetadas especificamente para suportar programação concorrente. Linguagens de propósito geral, como Java, suportam concorrência como um dos seus muitos objetivos de projeto.

Sistemas de Informação Programas projetados para instituições gerenciarem seus sistemas de informação provavelmente são os mais prolíficos do mundo. As corporações perceberam, na década de 1950, que o uso de computadores poderia reduzir bastante o tédio da manutenção de registros e melhorar a precisão e a confiabilidade do que eles poderiam realizar. Os sistemas de informação encontrados em corporações incluem o sistema de folha de pagamento, o sistema de contabilidade, os sistemas de marketing e as vendas on-line, os sistemas de estoque e produção, e assim por diante. Tais sistemas se caracterizam pela necessidade de processar grandes quantidades de dados (muitas vezes organizados nos chamados bancos de dados), mas requerem transformações relativamente simples sobre os dados que estão sendo processados.

Tradicionalmente, os sistemas de informação têm sido desenvolvidos em linguagens de programação como Cobol e SQL. Cobol foi projetada primeiramente no final da década de 1950 por um grupo de representantes da indústria que queriam desenvolver uma linguagem que fosse portátil em uma diversidade de diferentes arquiteturas de máquina. Cobol significa “*COmmon Business Oriented Language*” (Linguagem Comum Orientada aos Negócios), usa a língua inglesa como base da sua sintaxe e suporta um estilo de programação *imperativo*.

Programas em Cobol são construídos a partir de cláusulas, sentenças e parágrafos e, geralmente, tendem a ser mais prolixos que programas similares em outras linguagens. O objetivo era definir uma linguagem que fosse fácil para os programadores assimilarem. Se esse objetivo foi ou não atingido é algo que ainda está aberto a discussão. Apesar disso, Cobol se tornou rapidamente, e ainda continua sendo, a linguagem de programação mais amplamente usada para aplicações de sistemas de informação.

Em contraste, SQL (Pratt, 1990) emergiu na década de 1980 como uma ferramenta de programação *declarativa* para especificação de bancos de dados, geração de relatórios e recuperação de informações. SQL significa “*Structured Query Language*” (Linguagem de Consulta Estruturada) e é a linguagem predominante usada para a especificação e a recuperação de informações de bancos de dados relacionais. O modelo relacional de bancos de dados é amplamente usado, em parte por causa do seu forte fundamento matemático em álgebra relacional.

Mais recentemente, os negócios desenvolveram ampla gama de aplicações de comércio eletrônico. Estas aplicações muitas vezes usam um modelo “cliente-servidor” de projeto de programa, que interage com os usuários em locais remotos e fornece acesso simultâneo a um banco de dados compartilhado. Um bom exemplo desse modelo é um sistema de encomenda de livros on-line, no qual o banco de dados reflete o estoque de livros da empresa e a interação auxilia o usuário na pesquisa pelo banco de dados, na seleção dos livros e no processo de encomenda. A *programação orientada a evento* é essencial nessas aplicações, e os programadores combinam linguagens como Java, Perl, Python e SQL para implementá-las.

Sistemas e Redes Programadores de sistemas projetam e realizam a manutenção do software básico que executa os sistemas – componentes do sistema operacional, software de rede, compiladores e depuradores de linguagens de programação, máquinas virtuais e interpretadores, sistemas embarcados e de tempo real (em telefones celulares, ATMs, aeronaves etc.). Esses tipos de software estão intimamente relacionados com as arquiteturas de máquinas específicas, como a Intel/AMD x86 e a Apple/Motorola/IBM PowerPC.

A maioria desses programas é escrita em C, que permite que os programadores cheguem a um nível muito próximo do nível de linguagem de máquina. A programação de sistemas é geralmente feita com o uso do paradigma de projeto *imperativo*. Todavia, os programadores de sistemas também devem lidar com a programação *concorrente e orientada a eventos*, além de ter uma preocupação especial com a *correção* do programa.

Assim, o exemplo primário de uma linguagem de programação de sistemas é C, projetada no início da década de 1970, em parte para suportar a codificação do sistema operacional Unix. De fato, cerca de 95% do código do sistema Unix é escrito em C. C++ foi projetado por Bjarne Stroustrup na década de 1980 como uma extensão de C para fornecer novos recursos que suportariam a programação orientada a objeto.

O nome da linguagem de programação Ada vem de Ada Lovelace, que, acredita-se, pode ter sido a primeira pessoa a programar computadores. No início da década de 1800, ela trabalhava com o inventor do computador Charles Babbage. O desenvolvimento da Ada foi financiado pelo Departamento de Defesa dos Estados Unidos, cuja idéia original era ter uma única linguagem que suportasse todas as aplicações desse departamento, especialmente aplicações de controle e sistemas embarcados. Embora Ada nunca tenha atingido esse objetivo específico, seu projeto tem alguns recursos notáveis. Hoje, Ada oferece uma base robusta sobre a qual o compilador Spark disponibiliza ferramentas de suporte à correção de programas.

Linguagens de *script* são largamente usadas atualmente para uma diversidade de tarefas de sistema. Por exemplo, um programa *awk* pode ser projetado rapidamente para verificar a consistência em um arquivo de senhas de uma máquina Unix. Algumas das principais linguagens de *script* são *awk* (Kernighan e Pike, 1984), Perl (Wall et al., 1996b), Tcl/Tk (Ousterhout, 1994) e Python (Lutz, 2001). Tratamos das linguagens de *script* no Capítulo 12, no qual a programação em Perl é explorada em algum detalhe.

World Wide Web A área mais dinâmica para novas aplicações de programação é a Internet, que é o veículo que permite o comércio eletrônico e uma ampla gama de aplicações acadêmicas, governamentais e industriais. A noção de computação centrada na Web e, conseqüentemente, de programação centrada na Web é motivada por um modelo interativo no qual um programa permanece ativo continuamente esperando pelo próximo evento, respondendo a este evento e retornando a seu estado continuamente ativo.

Linguagens de programação que suportam a computação centrada na Web usam a programação *orientada a evento*, que promove a interação sistema-usuário. A computação centrada na Web também usa o paradigma *orientado a objeto*, já que diversas entidades que aparecem na tela do usuário são modeladas de forma mais natural como objetos que enviam e recebem mensagens. As linguagens de programação que suportam computação centrada na Web incluem Perl, PHP (Hughes, 2001), Visual Basic, Java e Python.

1.5 SOBRE O PROJETO DE LINGUAGEM

O projeto de linguagem de programação é um desafio enorme. Os projetistas de linguagem são as pessoas que criam um meio de linguagem que permite aos programadores resolver problemas complexos. Para atingirem seu objetivo os projetistas devem trabalhar dentro de diversas restrições práticas e adotar objetivos específicos que se combinam para fornecer um foco a esse desafio. Esta seção dá uma visão geral desses objetivos e dessas restrições de projeto.

1.5.1 Restrições de Projeto

Os seguintes elementos de configurações computacionais impõem importantes restrições para os projetistas de linguagem:

- Arquitetura
- Configuração técnica
- Padrões
- Sistemas legados

Arquitetura Linguagens de programação são projetadas para computadores. Este fato é tanto uma bênção quanto uma maldição para os projetistas de linguagem. É uma bênção porque uma linguagem bem projetada e implementada pode melhorar muito a utilidade do computador em um domínio de aplicação. É uma maldição porque a maioria dos projetos de computadores nas últimas décadas foram limitados pelas idéias de arquitetura do modelo clássico de von Neumann-Eckert, discutido anteriormente. Muitas linguagens, como Fortran, Cobol e C, estão de acordo com essa arquitetura, enquanto outras, como Lisp, não estão.

Por alguns anos se tornou interessante considerar a idéia de arquitetura computacional como derivada do projeto de linguagem, em vez de como uma precursora. Na década de 1960, Burroughs projetou a B5500, que tinha uma arquitetura de pilha especialmente apropriada para executar programas em Algol. Outro esforço produziu o tipo de máquinas Lisp, que emergiram no início da década de 1980. Essas máquinas eram configuradas de modo que os programas em Lisp fossem executados eficientemente nelas e tiveram certo grau de sucesso durante alguns anos. Entretanto, as arquiteturas de máquina Lisp foram ofuscadas no final da década de 1980 pelo advento das arquiteturas RISC (*Reduced Instruction Set Computer* – Computadores com Conjunto Reduzido de Instruções), nas quais programas em Lisp podiam ser implementados eficientemente.

Assim, quando consideramos as virtudes de diversas opções de projeto de linguagens, sempre ficamos restritos pela necessidade de implementar a linguagem eficientemente e de forma eficaz dentro das restrições impostas pelas variações atuais do modelo clássico de von Neumann. A noção de que um bom projeto de linguagem pode levar a uma arquitetura computacional radicalmente nova e comercialmente viável possivelmente não é provável.

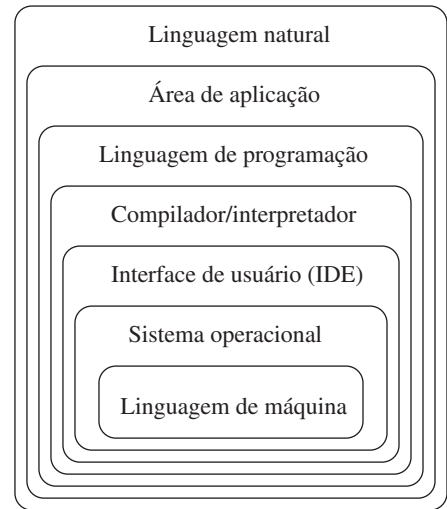
Configuração Técnica As linguagens são restritas não apenas pelos limites das arquiteturas de computadores, mas também devem satisfazer a outras restrições impostas pelas configurações técnicas nas quais elas são usadas: a área de aplicação, o sistema operacional, o IDE (*Integrated Development Environment* – Ambiente de Desenvolvimento Integrado), a rede e as outras preferências de uma determinada comunidade de programação. Por exemplo, Fortran é utilizado em determinadas plataformas por diferentes compiladores para se adaptar às necessidades de programadores científicos. Estes programadores trabalham em diversas profissões que usam seus estilos próprios de projeto de software, ferramentas e (acima de tudo) suas próprias linguagens naturais para comunicar-se entre eles. Essa visão do cenário complexo para o projeto de linguagens é resumida na Figura 1.3.

Algumas linguagens são intencionalmente mais de *uso geral* no seu projeto, tendo como objetivo ser úteis aos interesses de uma ampla gama de aplicações. Por exemplo, Ada (The Department of Defense, 1983) foi projetada para ser útil para todas as aplicações suportadas pelo Departamento de Defesa dos Estados Unidos, enquanto Cobol (Brown, 1977) foi projetada para suportar todas as aplicações orientadas a negócio. Embora Cobol tenha sido moderadamente bem-sucedida no seu objetivo, Ada o foi muito menos.

Outras linguagens são projetadas para serem de *propósito específico* na sua natureza. Por exemplo, Prolog (Clocksin e Mellish, 1997) foi projetada para servir aos interesses restritos do processamento de linguagem natural, às provas de teoremas e aos sistemas especializados. C foi projetada principalmente para suportar os interesses da programação de sistemas, embora tenha, desde então, sido adotada por uma faixa mais ampla de aplicações. Spark/Ada e JML foram projetadas, respectivamente, para suportar a prova formal de correção de programas em Ada e Java.

Padrões Quando uma linguagem de programação tem um uso suficientemente amplo entre programadores, geralmente se inicia um processo de *padronização*, ou seja, é feito um esforço para definir um padrão independente da máquina da linguagem à qual

Figura 1.3 Níveis de Abstração na Computação



todos os seus programadores devem aderir. A padronização da linguagem geralmente a estabiliza em diferentes plataformas e grupos de programação, viabilizando a portabilidade dos programas.

As duas maiores organizações que supervisionam e mantêm padrões de linguagens de programação são a American National Standards Institute (ANSI) e a International Standards Organization (ISO). Diversas linguagens foram padronizadas com o passar dos anos desde que o processo de padronização de linguagens começou. Algumas delas, com suas datas de padronização, são:

ANSI/ISO Cobol (2002)

ISO Fortran (2004)

ISO Haskell (1998)

ISO Prolog (2000)

ANSI/ISO C (1999)

ANSI/ISO C++ (2003)

ANSI/ISO Ada (2005)

ANSI Smalltalk (2002)

ISO Pascal (1990)

O processo de padronização de linguagens é complexo e lento, com muito tempo de envolvimento da comunidade e geralmente tem uma definição volumosa da linguagem-padrão como resultado.

A padronização de linguagens de programação é acompanhada da padronização de conjuntos de caracteres (por exemplo, os conjuntos ASCII e UNICODE) e bibliotecas (por exemplo, a C++ Standard Template Library) que suportam diretamente as atividades de programação.

O valor da padronização para a comunidade é que os projetistas de software e hardware participam do processo e se comprometem com suas implementações de compiladores e in-

terpretores para que estejam em conformidade com o padrão. Tal conformidade é essencial na manutenção da portabilidade em diferentes plataformas de hardware e compiladores.

Alguns têm argumentado que a padronização de linguagens é uma influência negativa porque inibe inovações no projeto de linguagens, ou seja, versões-padrão das linguagens tendem a durar muito tempo, perpetuando, dessa forma, a vida de características pobres junto aos seus recursos mais valiosos. Entretanto, os padrões ISO e ANSI são revisados a cada cinco anos, o que fornece certa proteção contra a obsolescência prolongada.

Mais informações sobre padrões específicos de uma linguagem e o próprio processo de padronização podem ser encontradas em www.ansi.org e www.iso.org.

Sistemas Legados É bem sabido que a maior parte do tempo de um programador é gasto na manutenção de *sistemas legados*. Tais sistemas são aqueles artefatos de software que foram projetados e implementados pela equipe anterior de programação, mas que são mantidos e atualizados pela equipe atual. A maior parte de código em sistemas legados está provavelmente escrita em Cobol, a linguagem de programação predominante para sistemas de informação nas últimas quatro décadas.

Para suportar a manutenção de código legado, versões atualizadas e melhoradas de linguagens antigas devem ser *compatíveis para trás* com seus predecessores, ou seja, os programas antigos devem continuar a ser compilados e executados quando novos compiladores forem desenvolvidos para a versão atualizada. Assim, todos os recursos semânticos e sintáticos, mesmo os que forem menos desejáveis a partir do ponto de vista estético, não podem ser retirados sem prejudicar a integridade do código legado.

Por esse motivo, linguagens de programação mais antigas se tornaram sobrecarregadas com recursos quando novas versões surgiram; as linguagens raramente se tornam mais compactas enquanto se desenvolvem. Isso é particularmente verdadeiro para Fortran, Cobol e C++, que foi projetada como uma verdadeira extensão de C para manter compatibilidade para trás com código legado.

O projeto de Java – embora muitos dos seus recursos lembrem C++ –, afastou-se dessa tradição. Como tema central, os projetistas de Java queriam liberar sua nova linguagem de ter que suportar os recursos menos desejáveis de C++, de forma que eles simplesmente os cortaram. O resultado foi uma linguagem mais compacta, pelo menos temporariamente, já que versões recentes de Java adicionaram muitos recursos novos sem retirar um conjunto comparativamente grande de recursos obsoletos. Talvez seja inevitável que quando qualquer linguagem amadurece se torne naturalmente mais carregada de recursos para atender às crescentes demandas do domínio das aplicações.

1.5.2 Resultados e Objetivos

À luz desses requisitos, somos levados a fazer duas perguntas importantes:

- 1 Como uma linguagem de programação surge e se torna bem-sucedida?
- 2 Quais são as características-chave que tornam uma linguagem de programação ideal?

Examinando rapidamente o passado, primeiro observamos que algumas linguagens de programação bem-sucedidas foram projetadas por indivíduos, outras, por comitês de toda uma indústria, e outras ainda foram o produto de um forte apoio por parte dos seus desenvolvedores comerciais. Por exemplo, Lisp e C++ foram projetadas principalmente por indivíduos (John McCarthy e Bjarne Stroustrup, respectivamente), enquanto

Algol, Cobol e Ada foram projetadas por comitês¹. PL/I, Java e C# são produtos dos seus desenvolvedores comerciais corporativos (IBM, Sun e Microsoft, respectivamente). Assim, não está claro se o processo de projeto – individual, comitê ou alavancagem corporativa – possui uma influência total sobre o sucesso de um projeto de linguagem.

Já que este estudo tem como objetivo preparar os leitores para avaliar e comparar linguagens de programação de modo geral, é importante termos um pequeno conjunto de características-chave por meio das quais você possa fazer isso. Nós as chamaremos de *objetivos de projeto*, já que têm servido como medidas eficazes de projetos bem-sucedidos de linguagens com o passar dos anos:

- Simplicidade e legibilidade
- Clareza nas ligações
- Confiabilidade
- Suporte
- Abstração
- Ortogonalidade
- Implementação eficiente

Simplicidade e Legibilidade Os programas devem ser fáceis de escrever. Também devem ser compreensíveis e fáceis de ler pelo programador mediano. Uma linguagem de programação ideal deve, então, suportar a escrita e a leitura. Além disso, deve ser fácil de aprender e ensinar.

Algumas linguagens, como Basic, Algol e Pascal, foram intencionalmente projetadas para facilitar a clareza de expressão. Basic, por exemplo, tinha um conjunto de instruções muito pequeno. Algol 60 tinha uma “linguagem de publicação” que fornecia um formato-padrão para a composição de programas que apareciam em artigos publicados. Pascal foi projetada explicitamente como uma linguagem para ensino, com recursos que facilitavam o uso de princípios da programação estruturada.

Outras linguagens foram projetadas para minimizar o número total de teclas pressionadas que era necessário para expressar um algoritmo ou a quantidade de armazenamento que o compilador requeria. Com certeza, os projetistas de APL e C valorizaram essas economias.

Clareza nas Ligações Um elemento de linguagem está *conectado* a uma propriedade quando essa propriedade é definida para ele. Uma boa linguagem deve ser muito clara sobre quando ocorre a ligação principal para cada elemento com sua propriedade. Aqui estão os principais tempos de ligação.

- Tempo de definição da linguagem: Quando a linguagem é definida, os tipos básicos de dados são ligados a rótulos especiais denominados *palavras reservadas*, que os representam. Por exemplo, números inteiros (*integers*) são ligados ao identificador `int`, e números reais (fracionários) são ligados a `float` na linguagem C.
- Tempo de implementação da linguagem: Quando o compilador ou interpretador da linguagem é escrito, os valores são ligados a representações de máquina. Por exemplo, o tamanho de um valor `int` em C é determinado no momento da implementação da linguagem.

1. No caso de Ada, o processo de projeto também teve um elemento de competição – diversos projetos competindo foram avaliados, e Ada surgiu como a linguagem mais propícia para satisfazer às necessidades do Departamento de Defesa dos Estados Unidos da América.

- **Tempo de escrita do programa:** Quando os programas são escritos em algumas linguagens, os nomes de variáveis são ligados a tipos, os quais permanecem ligados com esses nomes por toda a execução do programa. Por exemplo, uma variável pode ser ligada ao seu tipo quando ela é declarada, como na declaração

```
int x;
```

que liga a variável `x` ao tipo `int`.

- **Tempo de compilação:** Quando os programas são compilados, os comandos e as expressões desses programas são ligados a seqüências de instruções em linguagem de máquina equivalente.
- **Tempo de carga do programa:** Quando o código de máquina é carregado, as variáveis estáticas são atribuídas a endereços fixos de memória, a pilha de tempo de execução é alocada a um bloco de memória, assim como o próprio código de máquina.
- **Tempo de execução do programa:** Quando os programas estão sendo executados, as variáveis são ligadas a valores, como na execução da atribuição `x = 3`.

Às vezes, um elemento pode ser ligado a uma propriedade em qualquer uma dentre diversas alternativas de tempos nessa série contínua. Por exemplo, a ligação de um valor com uma constante pode ser feita em tempo de compilação/carga de um programa ou no início da execução. Quando tais escolhas são possíveis, a noção de *ligação precoce* significa simplesmente que um elemento é ligado a uma propriedade o mais rápido possível (em vez de mais tarde) nessa série de tempo. A *ligação tardia* significa um atraso até a última oportunidade possível.

Como veremos no Capítulo 4, a ligação precoce leva a uma melhor detecção de erros e geralmente é menos custosa. Todavia, ligações tardias levam a uma maior flexibilidade de programação (conforme ilustrado no Capítulo 14). De modo geral, um projeto de linguagem deve considerar todas essas alternativas, e decisões sobre ligações são tomadas, em última instância, pelo projetista da linguagem.

Confiabilidade O programa se comporta da mesma forma toda vez que ele é executado com os mesmos dados de entrada? Ele se comporta da mesma forma quando é executado em diferentes plataformas? O seu comportamento pode ser especificado independentemente de modo a permitir a sua verificação formal (ou informal)?

Particularmente pertinente a essas questões é a necessidade de se projetar mecanismos apropriados de manipulação de exceções na linguagem. Além disso, linguagens que restrinjam o uso de aliases e vazamento de memória, que suportem tipagem forte, tenham sintaxe e semântica bem definidas e que suportem a verificação e validação de programas têm uma vantagem nessa categoria.

Suporte Uma boa linguagem de programação deve ser facilmente acessível por alguém que queira aprendê-la e instalá-la no seu próprio computador. De forma ideal, seus compiladores devem ser de domínio público, em vez de serem propriedade de uma corporação e caros para se obter. A linguagem deve ser implementada em múltiplas plataformas. Cursos, livros-texto, tutoriais e um grande número de pessoas familiarizadas com a linguagem são vantagens que ajudam a preservar e estender a vitalidade de uma linguagem.

Questões relacionadas a custo podem ser de maior preocupação para alunos e programadores individuais, em vez de para funcionários governamentais e corporativos, cujos custos de software são geralmente cobertos por suas empresas. A história das linguagens de programação tem apresentado sucesso em ambos os lados. Por exemplo, C, C++ e Java são linguagens não proprietárias, disponíveis no domínio público para uma ampla variedade de plataformas. Por outro lado, C# e Eiffel são

linguagens suportadas por empresas cujo uso é restringido pelo seu custo e as plataformas/IDEs nas quais elas são implementadas.

Abstração A abstração é um aspecto fundamental do processo de projeto de programas. Os programadores gastam muito tempo construindo abstrações, tanto de dados quanto procedurais, para explorar a reutilização de código e evitar reinventá-lo. Uma boa linguagem de programação suporta tão bem a abstração de dados e procedural que é a ferramenta de projeto preferida na maioria das aplicações.

Bibliotecas que acompanham linguagens modernas de programação comprovam a experiência acumulada de programadores na construção de abstrações. Por exemplo, as bibliotecas de classes Java contêm implementações de estruturas de dados básicos (como vetores e pilhas) que, em linguagens anteriores, tinham de ser projetadas explicitamente pelos próprios programadores. Com qual frequência nós reinventamos um algoritmo de ordenação ou uma estrutura de dados de lista encadeada que provavelmente já tenha sido implementado milhares de vezes antes?

Ortogonalidade Uma linguagem é dita *ortogonal* se os seus comandos e recursos são construídos sobre um conjunto pequeno e mutuamente independente de operações primitivas. Quanto mais ortogonal uma linguagem, menos regras excepcionais são necessárias para escrever programas corretos. Assim, programas em uma linguagem ortogonal muitas vezes tendem a ser mais simples e mais claros do que aqueles em uma linguagem não-ortogonal.

Como um exemplo de ortogonalidade, considere a passagem de argumentos em uma chamada de função. Uma linguagem integralmente ortogonal permite que qualquer tipo de objeto, incluindo uma definição de função, seja passado como argumento. Veremos exemplos disso na nossa discussão sobre programação funcional no Capítulo 14.

Outras linguagens restringem os tipos de objetos que podem ser passados em uma chamada. Por exemplo, a maioria das linguagens imperativas não permite que definições de funções sejam passadas como argumentos e, portanto, não são ortogonais em relação a isso.

A ortogonalidade tende a se correlacionar com a simplicidade conceitual, já que o programador não precisa manter muitas regras excepcionais na sua cabeça. Alan Perlis coloca isso da seguinte maneira:

É melhor fazer cem funções operarem em uma estrutura de dados do que dez funções em dez estruturas de dados.

Por outro lado, a não-ortogonalidade, muitas vezes está relacionada à eficiência porque suas regras excepcionais eliminam as opções de programação que consumiriam tempo ou espaço.

Implementação Eficiente Os recursos e as construções de uma linguagem devem permitir uma implementação prática e eficiente em plataformas contemporâneas.

Como um contra-exemplo, Algol 68 era um projeto de linguagem elegante, mas suas especificações eram tão complexas que era (quase) impossível implementá-las efetivamente. Versões iniciais de Ada foram criticadas pelas suas características de tempo de execução ineficientes, já que Ada foi projetada, em parte, para suportar programas que seriam executados em “tempo real”. Programas embarcados em sistemas como aeronaves tinham de responder imediatamente a uma mudança brusca nos valores de entrada, como vento e velocidade. Programas em Ada ficavam na interface entre os sensores que forneciam as leituras e os mecanismos que deveriam responder a elas. Implementações iniciais de Ada ficavam longe desses ambiciosos objetivos de desempenho. Os críticos mais severos do desempenho de Ada ficaram conhecidos por dizer “Bom, existe ‘tempo real’ e ‘tempo de Ada’”!

Implementações iniciais de Java receberam críticas por esse mesmo motivo, embora refinamentos recentes no sistema de compilação Java da Sun tenham melhorado seu desempenho em tempo de execução.

1.6 COMPILADORES E MÁQUINAS VIRTUAIS

Qualquer implementação de uma linguagem de programação requer que os programas nessa linguagem sejam analisados e então traduzidos em um formato que possa ser:

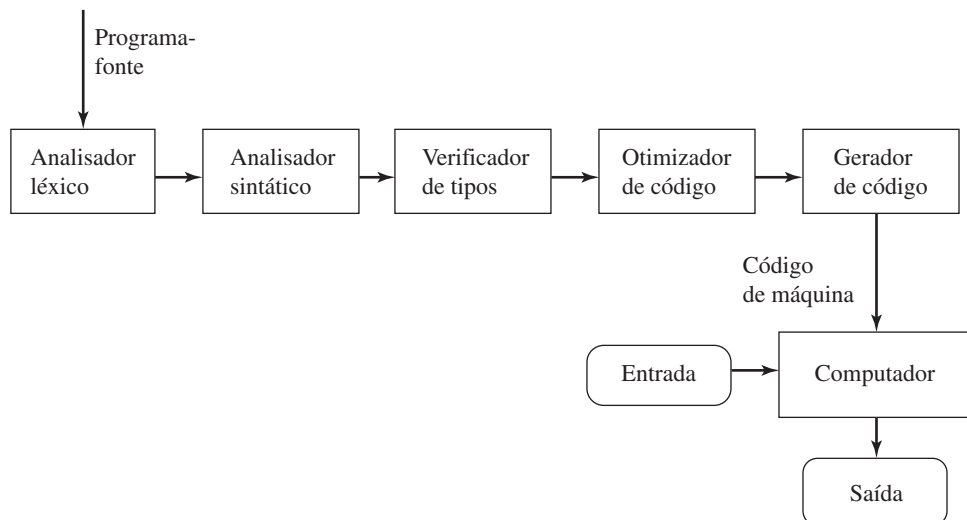
- 1 Executado por um computador (por exemplo, uma “máquina real”) ou
- 2 Executado por um interpretador (por exemplo, um software que simule uma “máquina virtual” e seja executado em uma máquina real).

A tradução do primeiro tipo é, muitas vezes, chamada de *compilação*, enquanto a tradução do segundo tipo é chamada *interpretação*.

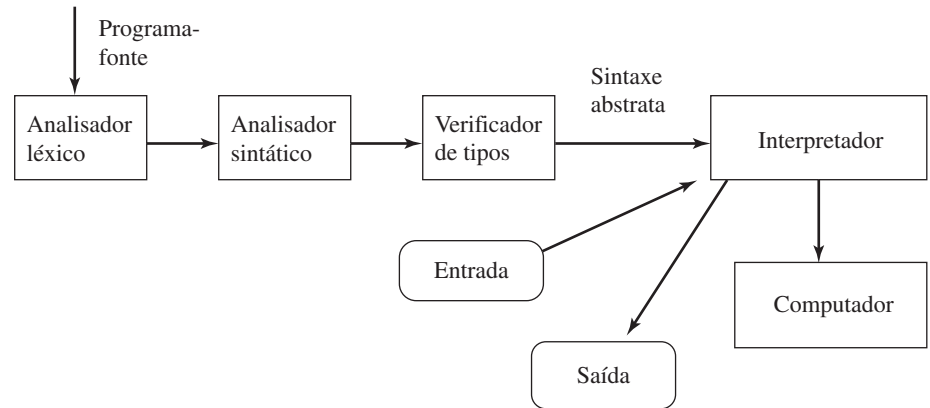
Compiladores O processo de *compilação* traduz um programa-fonte para a linguagem de um computador. Depois, o *código de máquina* resultante pode ser executado nesse computador. Esse processo é mostrado na Figura 1.4. Por exemplo, Fortran, Cobol, C e C++ são linguagens compiladas típicas.

Os cinco passos do processo de compilação são análise léxica, análise sintática, verificação de tipos, otimização de código e geração de código. Os três primeiros passos se relacionam a encontrar e relatar erros para o programador. Os dois últimos passos se relacionam à geração de código de máquina eficiente para ser executado no computador-alvo.

O código de máquina de um programa compilado se combina com sua entrada para ser executado em um passo separado que segue a compilação. Erros em tempo de execução geralmente são rastreáveis no programa-fonte por meio do uso de um depurador.



| Figura 1.4 O Processo de Compilação e Execução



| **Figura 1.5** Máquinas Virtuais e Interpretadores

Teremos algumas oportunidades de explorar os três primeiros passos da compilação nos Capítulos 3 e 6. Entretanto, os assuntos de geração e otimização de código são normalmente estudados em um curso de compilador, e não serão abordados neste texto.

Máquinas Virtuais e Interpretadores Outras linguagens são implementadas por meio do uso de um processo *interpretativo*, conforme mostrado na Figura 1.5. Aqui, o programa fonte é traduzido para uma forma abstrata intermediária, que é então executada de modo interpretado. Lisp e Prolog, por exemplo, são, muitas vezes, implementadas usando interpretadores (embora compiladores para essas linguagens também existam).

Como a Figura 1.5 sugere, os primeiros três passos de um compilador também acontecem em um interpretador. Entretanto, a representação abstrata do programa que sai desses três passos se torna o objeto de execução de um interpretador. O próprio interpretador é um programa que executa os passos do programa abstrato enquanto é executado em uma máquina real. O interpretador geralmente é escrito em uma linguagem distinta da que está sendo interpretada.

Às vezes, uma linguagem é projetada de modo que o compilador seja escrito uma só vez, direcionando o código para uma *máquina virtual* abstrata, e então essa máquina virtual é implementada por um interpretador em cada uma das diferentes máquinas reais. Este é o caso de Java, cuja máquina abstrata foi chamada de Java Virtual Machine (JVM) (Lindholm e Yellin, 1997). Ao fazer essa escolha, os projetistas da linguagem Java desistiram um pouco da eficiência (já que código interpretado geralmente requer mais recursos do que código de máquina) em favor de flexibilidade e portabilidade, ou seja, qualquer mudança na especificação da linguagem Java pode ser implementada pela alteração de um único compilador, em vez de uma família de compiladores.

Uma vantagem importante da compilação sobre a interpretação é que o desempenho em tempo de execução de um programa compilador geralmente é mais eficiente (em tempo e espaço) do que seu desempenho interpretado. Por outro lado, a qualidade da interação com o sistema que o programador desfruta pode ser melhor com um interpretador do que com um compilador. Já que o desenvolvimento de programas é interativo, segmentos individuais de programas (funções) podem ser testados na medida em que são

projetados, em vez de esperar até que o programa inteiro no qual eles devem ser inseridos seja completado².

Versões posteriores de Java obtiveram de volta um pouco da eficiência perdida pela inserção de um compilador *just-in-time* (JIT) da JVM. Esse recurso permite que o *byte code* da JVM seja traduzido durante a execução em código nativo da máquina hospedeira antes que seja executado.

O conceito de máquina virtual é valioso por outros motivos que podem compensar sua inerente perda de eficiência. Por exemplo, é oportuno implementar uma linguagem e seu interpretador usando uma máquina virtual existente, visando ao projeto ou à experimentação com a própria linguagem. De forma semelhante, é útil estudar fundamentos e paradigmas de projeto de linguagens usando um *interpretador* disponível que facilite a experimentação e a avaliação de programas nessa linguagem.

O conceito de máquina virtual possui um valor prático imediato neste estudo, ou seja, você terá acesso a um interpretador para um pequeno subconjunto C denominado *Clite*. O uso do Clite facilita muito o nosso trabalho porque elimina detalhes específicos de máquina que muitas vezes podem esconder os princípios e as idéias que seriam ensinadas. Por exemplo, no estudo da sintaxe da linguagem, você pode exercitar o interpretador Clite para explorar a estrutura sintática de diferentes elementos da linguagem como expressões aritméticas e laços.

1.7 RESUMO

O estudo de linguagens de programação inclui princípios, paradigmas e tópicos especiais. Os princípios são estudados tanto conceitualmente quanto na prática, por intermédio do interpretador Clite.

O domínio de um ou mais novos paradigmas – programação imperativa, orientada a objeto, funcional ou lógica – também é importante para este estudo. Essa atividade nos ajuda a apreciar uma faixa mais ampla de aplicações computacionais e descobrir abordagens para a resolução de problemas com os quais ainda não estamos familiarizados.

A investigação de um ou mais tópicos especiais – manipulação de eventos, concorrência ou correção – nos permite examinar com cuidado três características específicas de projeto de linguagem e os desafios de programação que envolvem sua utilização efetiva.

No total, esperamos que este estudo ajude a ampliar sua visão e suas habilidades técnicas no vasto cenário das linguagens de programação. Em especial, você deve esperar obter:

- A apreciação do uso de ferramentas práticas para examinar os princípios de projeto de linguagem.
- A apreciação do valor de diferentes paradigmas de programação que sejam especialmente poderosos em domínios específicos de aplicações.
- Experiências em laboratório com novas linguagens e ferramentas de projeto, tanto para testar os princípios quanto para dominar novas técnicas de resolução de problemas.

Para apoiar este estudo você pode ocasionalmente visitar o site do livro, que traz também outros materiais de apoio pedagógico e de aprendizado.

2. Entretanto, o desenvolvimento de depuradores e IDEs modernos para linguagens compiladas tem neutralizado substancialmente essa vantagem nos últimos anos.

EXERCÍCIOS

- 1.1 Uma pesquisa on-line sobre “linguagens de programação” produzirá *links* para fontes de informação confiáveis de todas as linguagens de programação importantes do passado e do presente. Para cada uma das seguintes linguagens, use a Internet para aprender algo sobre ela. Escreva, com suas próprias palavras, um breve resumo (um parágrafo) das suas características diferenciais, assim como sua relação histórica com outras linguagens que a precederam ou seguiram.
 - (a) Eiffel
 - (b) Perl
 - (c) Python
- 1.2 Dê um exemplo de um comando em C, C++ ou Java, que seja especialmente ilegível. Reescreva esse comando em um estilo mais legível. Por exemplo, você já viu a expressão `A[i++]` em um programa C/C++?
- 1.3 Código ilegível não é algo exclusivo de C, C++ e Java. Considere as seguintes opiniões fortemente defendidas sobre as fraquezas de determinadas linguagens nas últimas quatro décadas ou mais:

“É praticamente impossível ensinar boa programação a alunos que tiveram uma exposição anterior a BASIC; como programadores em potencial, eles ficam mentalmente mutilados além da esperança de regeneração.” E. Dijkstra

“O uso de COBOL enfraquece a mente; seu ensinamento, portanto, deve ser considerado como ofensa criminal.” E. Dijkstra

“APL é um equívoco levado à perfeição. É a linguagem do futuro para as técnicas de programação do passado: ela cria uma nova geração de programadores preguiçosos.” E. Dijkstra

“Não existe agora, e nem existirá, uma linguagem de programação na qual seja difícil escrever programas ruins.” L. Flon

 - (a) Dijkstra parece não ter muita consideração com Basic, Cobol ou APL. Entretanto, ele *teve* muita consideração com Algol e suas sucessoras. Faça uma leitura eficaz na Internet para determinar quais as características gerais que Algol possuía que a tornava superior a linguagens como Basic, Cobol e APL.
 - (b) O que Flon quer dizer com essa última declaração? As linguagens de programação são inerentemente imperfeitas? Ou ele está sugerindo que os programadores são inerentemente ineptos? Ou existe uma interpretação intermediária a essas duas? Explique.
- 1.4 Dê uma característica de C, C++ ou Java que ilustre a ortogonalidade. Dê uma característica diferente da discutida no texto que ilustre a não-ortogonalidade.
- 1.5 Duas implementações diferentes de uma linguagem são *incompatíveis* se houver programas que sejam executados de forma diferente (dão resultados diferentes) sob uma implementação em relação à outra. Após ler em um site e em outras fontes sobre versões anteriores de Fortran, você pode determinar se ela tinha versões incompatíveis? De que forma específica (tipo de comando, tipo de dado etc.) esta incompatibilidade aparecia? O que pode ser feito para eliminar incompatibilidades entre duas implementações diferentes de uma linguagem?
- 1.6 O trabalho de padronização para a linguagem C começou em 1982 com um grupo de trabalho ANSI, e o primeiro padrão C foi concluído em 1989. Este trabalho foi posteriormente aceito como um padrão ISO em 1990 e continua sendo até hoje. Leia o suficiente na Internet para determinar quais alterações significativas foram feitas no padrão C desde 1990.

- 1.7 Encontre o padrão C++ na Internet. O que significa *não conformidade* quando o padrão discute uma característica da linguagem suportada por um determinado compilador? Para o compilador C++ no seu computador, há características que não possuem conformidade?
- 1.8 Após aprender o suficiente no site de Java (java.sun.com) e em outras fontes, o que você pode dizer sobre a situação do trabalho de padronização de Java pela ANSI e pela ISO neste momento?
- 1.9 Encontre a versão Python 2.4 na Internet. Quais são as características novas que essa versão adiciona a Python 2.3? Quais características antigas da Python são eliminadas pela versão mais nova, se houver alguma?
- 1.10 Compare duas linguagens que você conheça usando os objetivos do projeto de linguagem destacados na Seção 1.5.2. Para cada objetivo, determine qual das duas linguagens satisfaz melhor esse objetivo e justifique sua conclusão com um exemplo. Ou seja, na comparação de C e Java você poderia concluir que C possui implementação mais eficiente porque ele compila em código nativo em vez de em um interpretador.